

GIFT OF THE GIVERS

Portfolio of Evidence — APPR6312POE

Mentor Likheli

Tlogelo Molele

Katlego Motsoaledi

Mmakwena Masenya

Live app: <https://giftofthegiverspoe-dxewg7fhdqaqxg4.canadacentral-01.azurewebsites.net/>

Repository: https://github.com/XTROiD-00/gift_of_the_givers

Section A: Agile Project Planning with Azure Boards

1.1 Setting Up the Azure Boards Project

An Azure DevOps project was created for the Gift of the Givers disaster-relief application. Azure Boards was configured to support Agile development, allowing the team to manage responsibilities, requirements, tasks and sprint progress in one central location.

Group Members and Roles

Each group member was added to the Azure DevOps project and assigned responsibilities according to their role within the development team.

Role	Name	Responsibility
Project Lead	Mentor Lukheli	Coordinates the team, manages sprints and monitors progress
Developer	Mmakwena Masenya	Designs and develops the application's user interface
Database Designer	Katlego Motsoaledi	Designs and manages the Azure SQL database
Quality Reviewer	Tlogelo Molele	Tests functionality and identifies defects

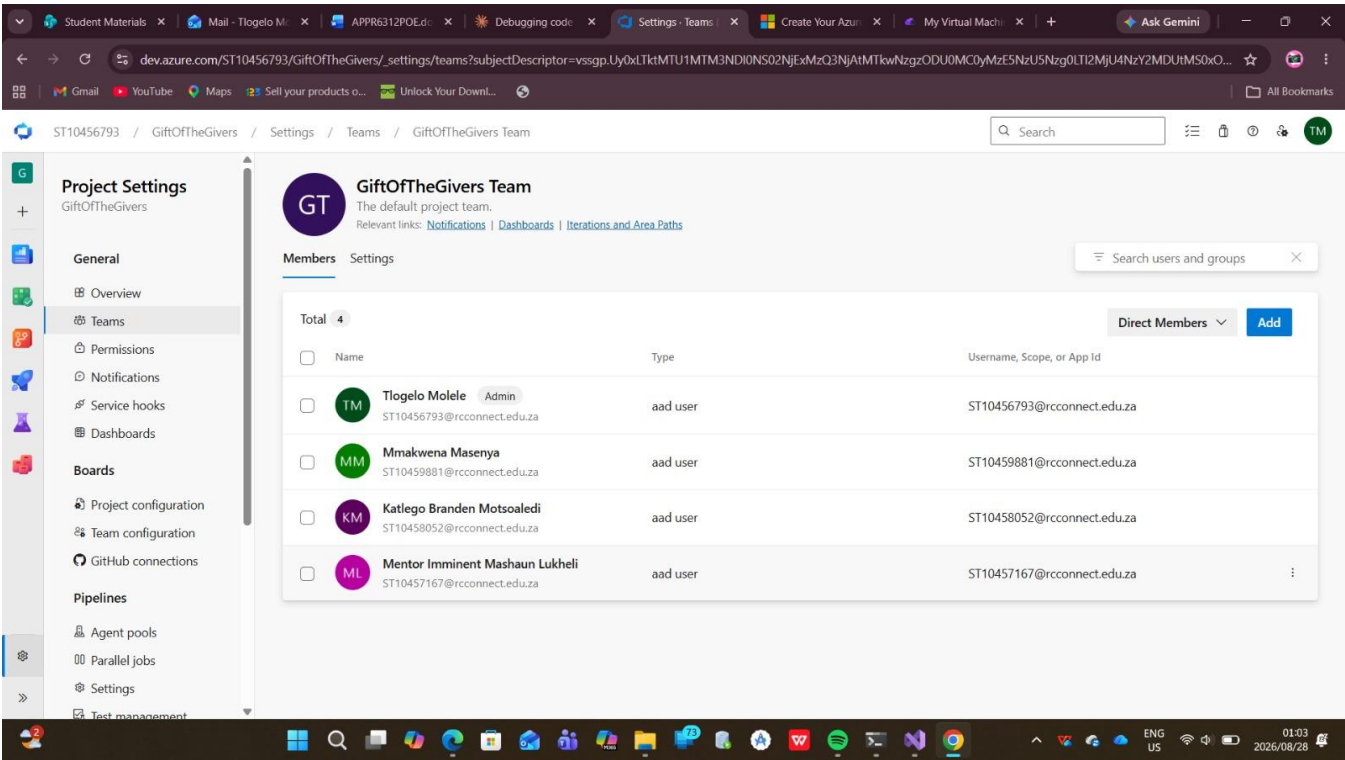


Figure 1: Azure DevOps team members

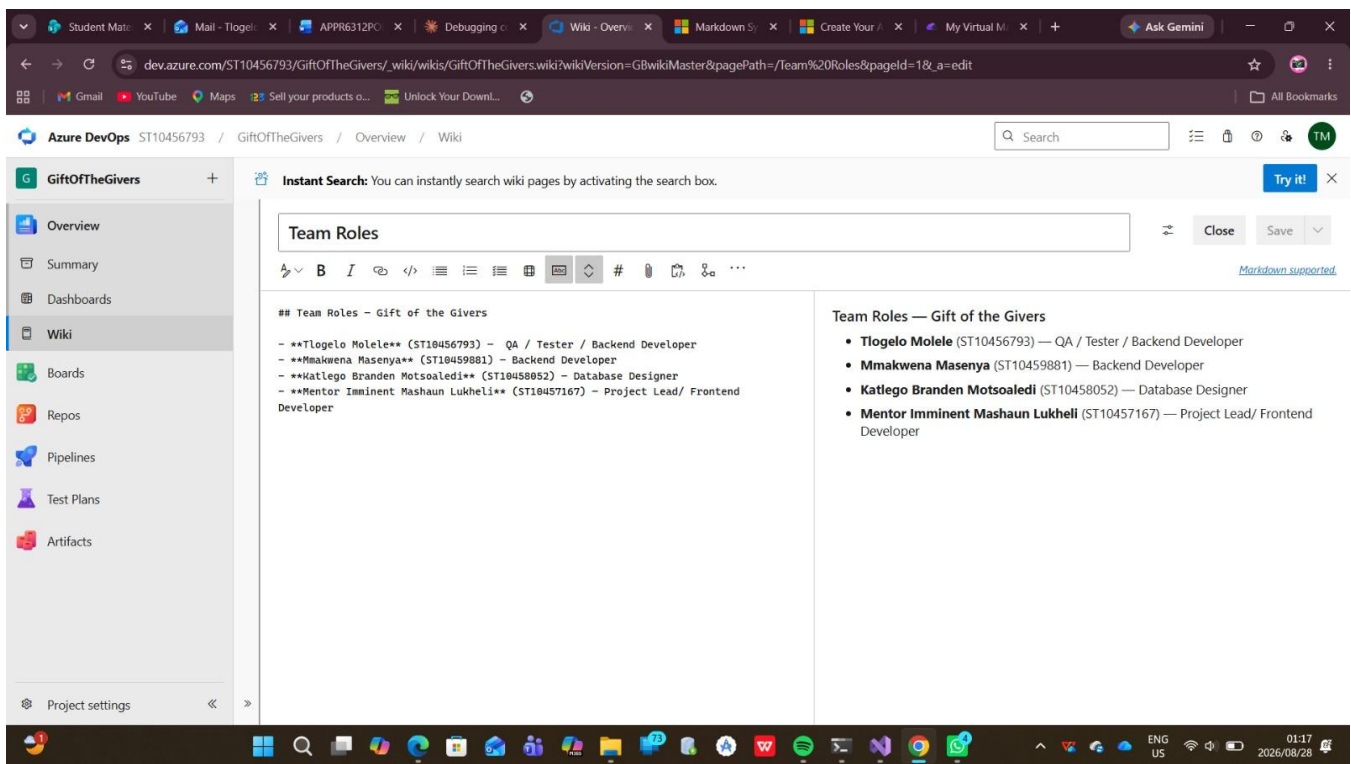


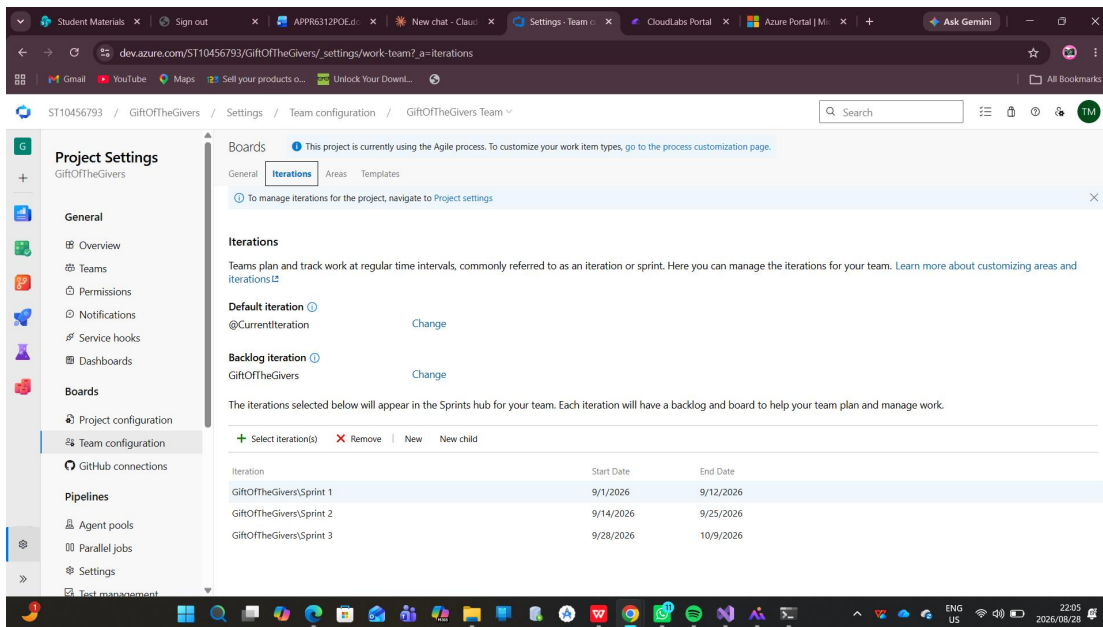
Figure 2: Team roles documented in the Azure DevOps wiki

Having clearly defined roles prevents duplication of work and ensures that each team member understands their responsibilities.

Iteration Paths

The project was divided into five sprints, tracked as iteration paths in Azure Boards:

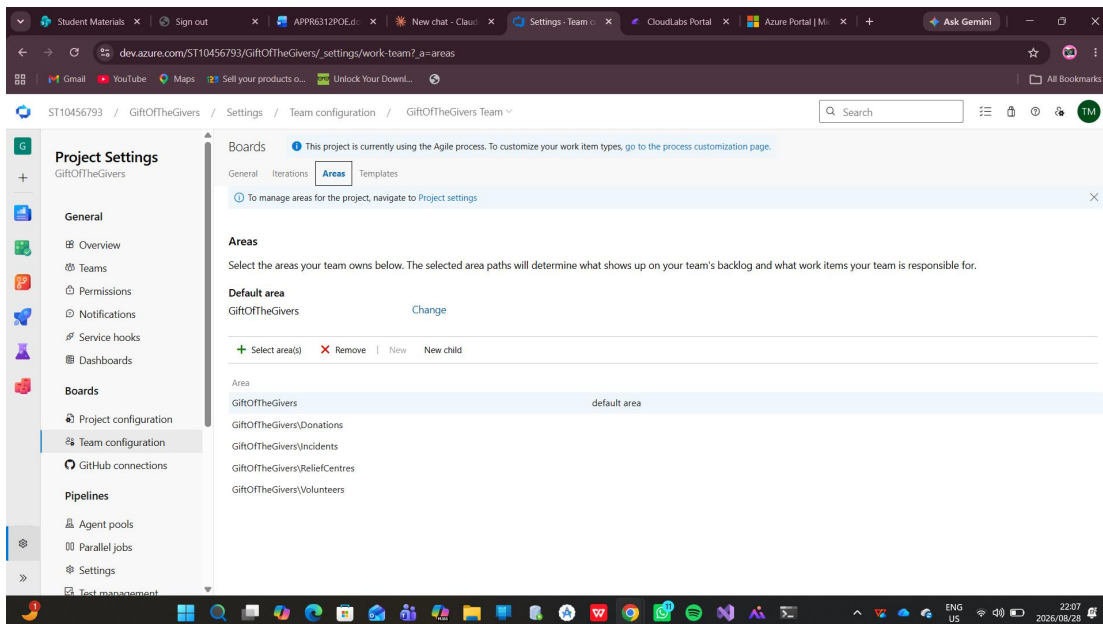
- Sprint 1 — Prototype: basic application structure, login and dashboard.
- Sprint 2 — Volunteer Management: volunteer registration and management.
- Sprint 3 — Donation Management: recording and managing donations.
- Sprint 4 — Relief Project Management: creating and managing relief projects.
- Sprint 5 — Testing and Improvements: testing, fixing defects and improving the application.



Area Paths

The application was split into functional areas using area paths, so the team could organise work according to different parts of the application:

- User Management
- Volunteer Management
- Donation Management
- Relief Project Management



Collaboration

Azure Boards supports Agile collaboration because all team members can see the work that needs to be completed, who is responsible for it, and its current progress. Team members update tasks, communicate

progress, and move work items between stages such as To Do, In Progress and Done. This provides transparency and helps the team identify delays or incomplete work early.

1.2 Capturing Requirements as Epics, User Stories and Tasks

Requirements were organised into Epics, User Stories and Tasks. This breaks large requirements down into smaller, well-defined pieces of work that the development team can understand and complete.

Epic 1: User Management

User Story 1: User Registration

As a new user, I want to create an account so that I can access the disaster-relief application.

Tasks:

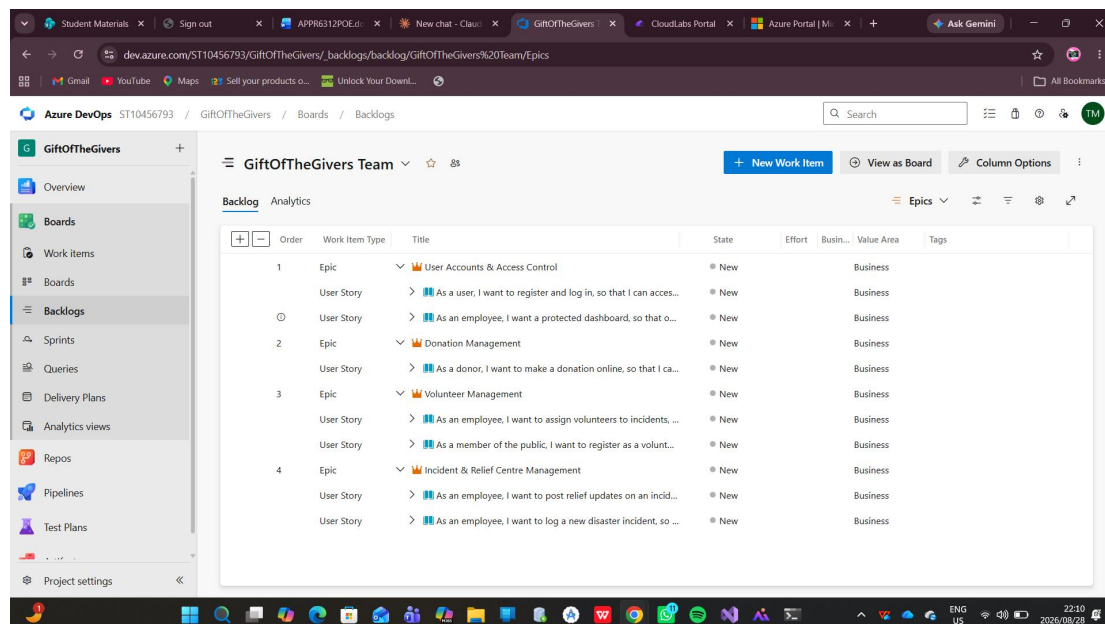
- Design the registration page.
- Create the Users database table.
- Implement the registration functionality.
- Validate user information.
- Store the user securely in the database.
- Test user registration.

User Story 2: User Login

As a registered user, I want to log into the system so that I can access features based on my role.

Tasks:

- Design the login page.
- Implement authentication.
- Validate login credentials.
- Implement role-based access.
- Test successful and unsuccessful login attempts.



Epic 2: Volunteer Management

User Story 1: Volunteer Registration

As a volunteer, I want to register my details so that Gift of the Givers can identify and manage available volunteers.

Tasks:

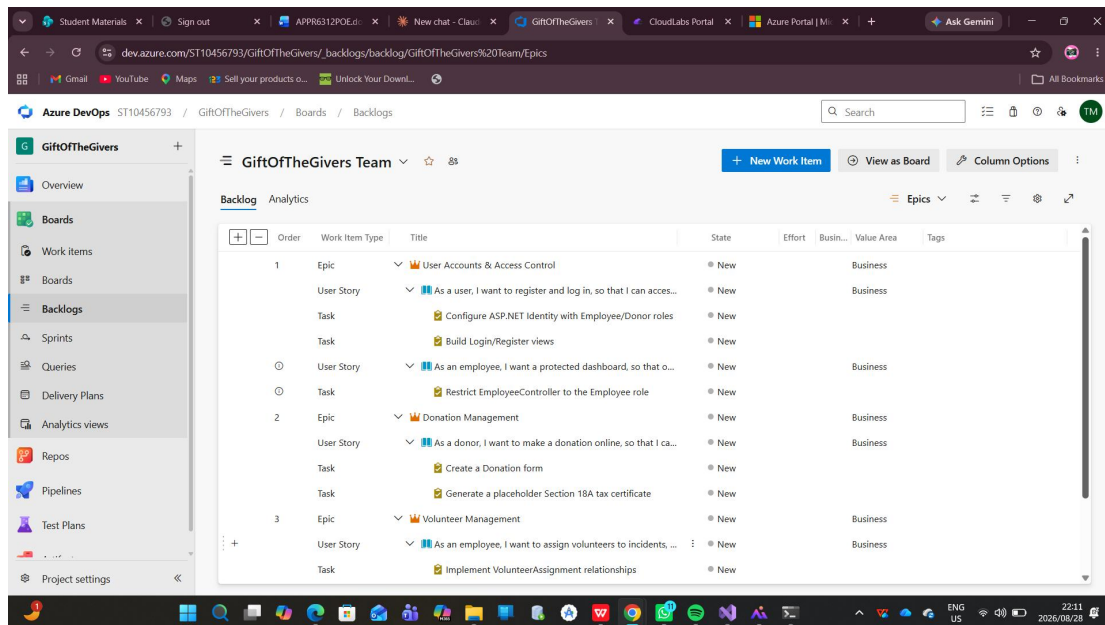
- Design the volunteer registration form.
- Create the Volunteer database table.
- Implement volunteer registration.
- Validate volunteer information.
- Save volunteer information to Azure SQL.
- Test the registration process.

User Story 2: View Volunteers

As a coordinator, I want to view registered volunteers so that I can identify volunteers available for disaster-relief activities.

Tasks:

- Create the volunteer list page.
- Retrieve volunteer records from the database.
- Display volunteer information.
- Add search/filter functionality.
- Test volunteer retrieval.



Epic 3: Donation Management

User Story 1: Record Donation

As a donor, I want to record my donation so that Gift of the Givers can keep an accurate record of available resources.

Tasks:

- Design the donation form.

- Create the Donations database table.
- Implement donation recording.
- Validate donation information.
- Save the donation to Azure SQL.
- Test donation recording.

User Story 2: View Donations

As an administrator, I want to view donations so that I can monitor the resources received by the organisation.

Tasks:

- Create the donations page.
- Retrieve donation records.
- Display donation information.
- Add filtering by date or type.
- Test the donation list.

Epic 4: Relief Project Management

User Story 1: Create Relief Project

As a relief coordinator, I want to create a relief project so that disaster-relief activities can be organised and tracked.

Tasks:

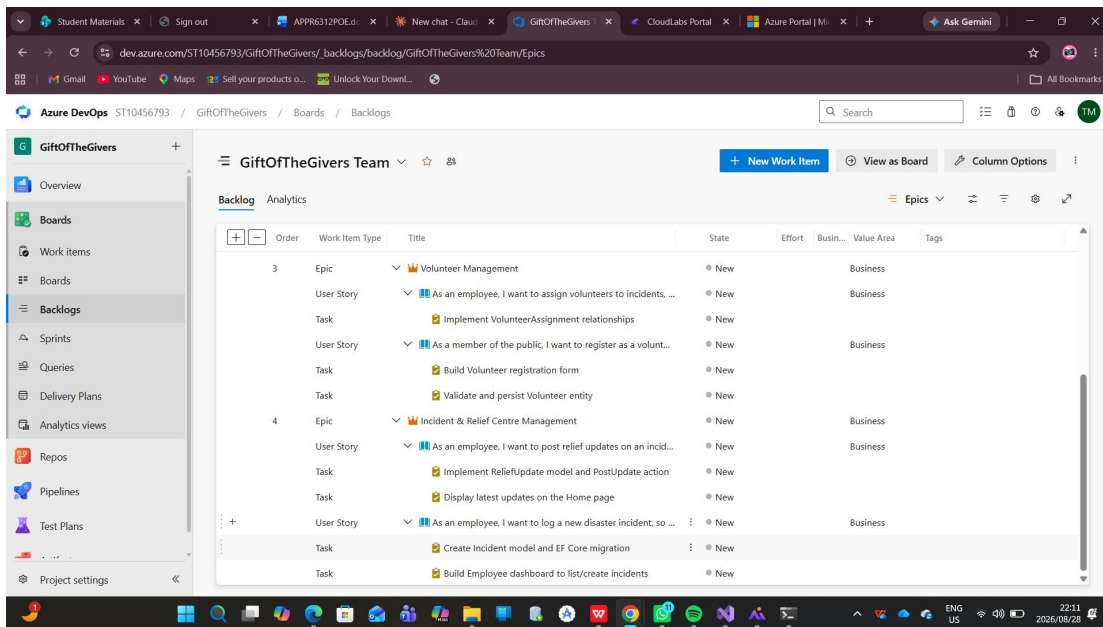
- Design the relief project form.
- Create the ReliefProjects table.
- Implement project creation.
- Validate project information.
- Store the project in Azure SQL.
- Test project creation.

User Story 2: Assign Volunteers

As a relief coordinator, I want to assign volunteers to relief projects so that available resources can be effectively managed.

Tasks:

- Create volunteer assignment functionality.
- Select an available volunteer.
- Select a relief project.
- Store the assignment.
- Prevent invalid or duplicate assignments.
- Test volunteer assignment.



Prioritisation

Work items were prioritised according to their importance to the first working version of the application.

High priority:

- User registration
- User login
- Volunteer registration
- Donation recording
- Relief project creation

Medium priority:

- Volunteer searching
- Donation filtering
- Volunteer assignment
- Project management

Lower priority:

- Advanced reporting
- Additional filtering
- Future improvements

The highest-priority features were required first because they form the foundation of the application. Additional functionality could then be introduced in later sprints.

1.3 Sprint Organisation

Sprint 1 — Application Prototype

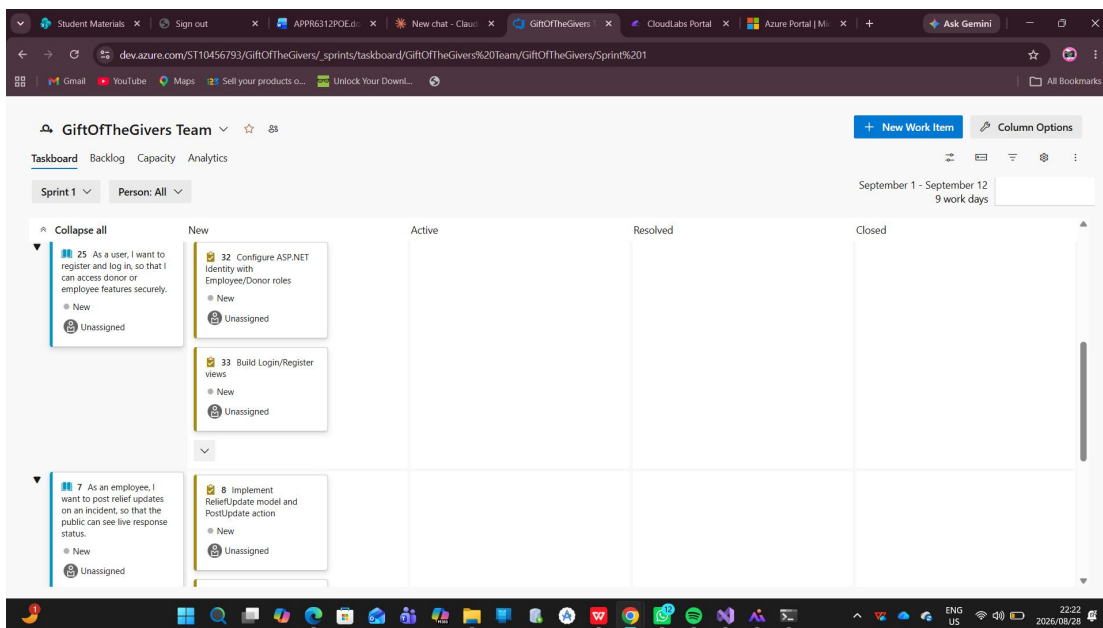
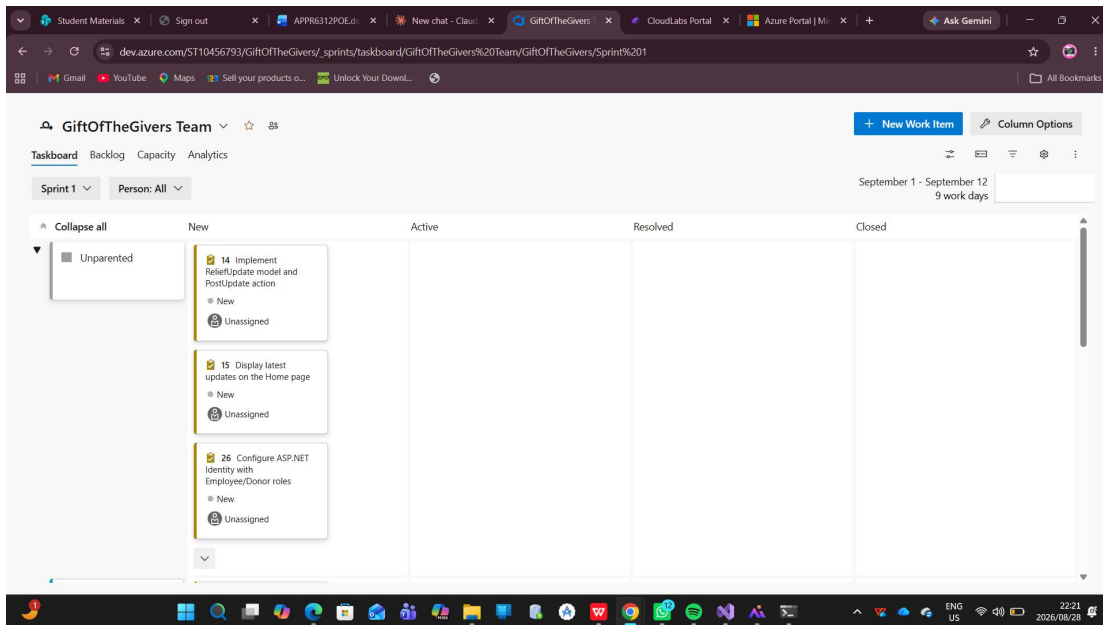
Goal: Create the basic working application.

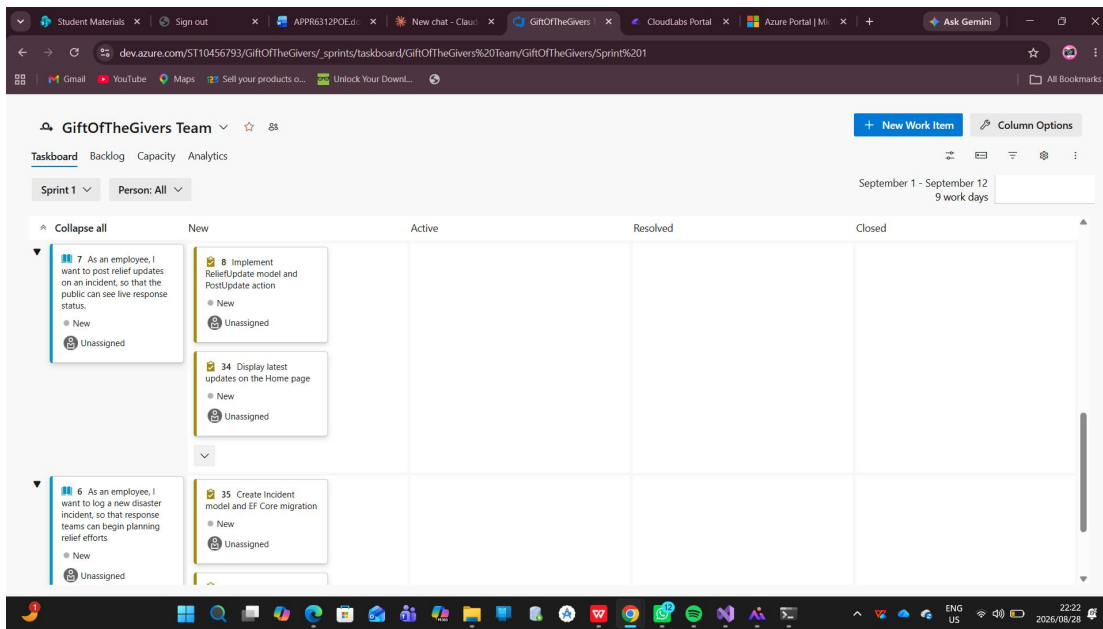
User Stories:

- User registration

- User login
- Basic dashboard

Deliverable: A working prototype containing authentication and the basic application structure.





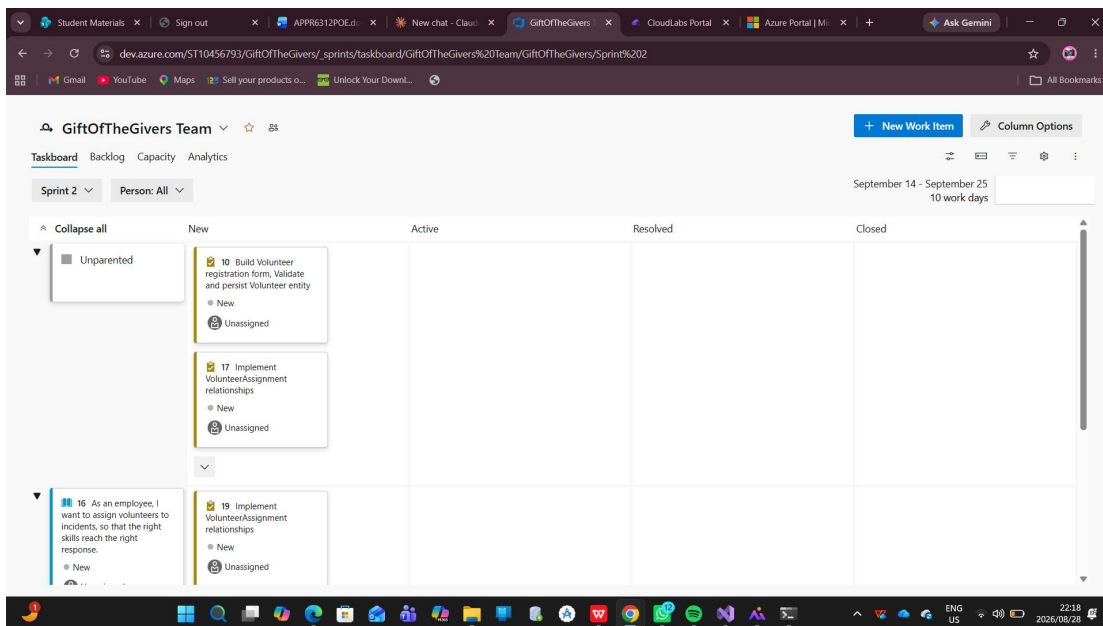
Sprint 2 — Volunteer Management

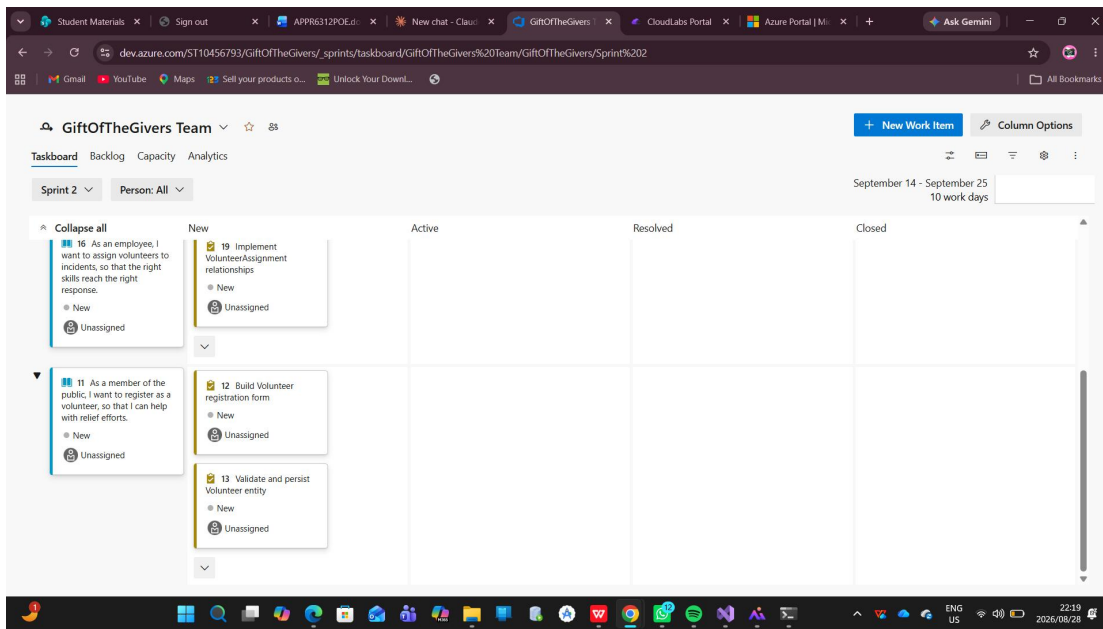
Goal: Allow the organisation to manage volunteers.

User Stories:

- Volunteer registration
- View volunteers
- Search volunteers

Deliverable: A functional volunteer-management module.





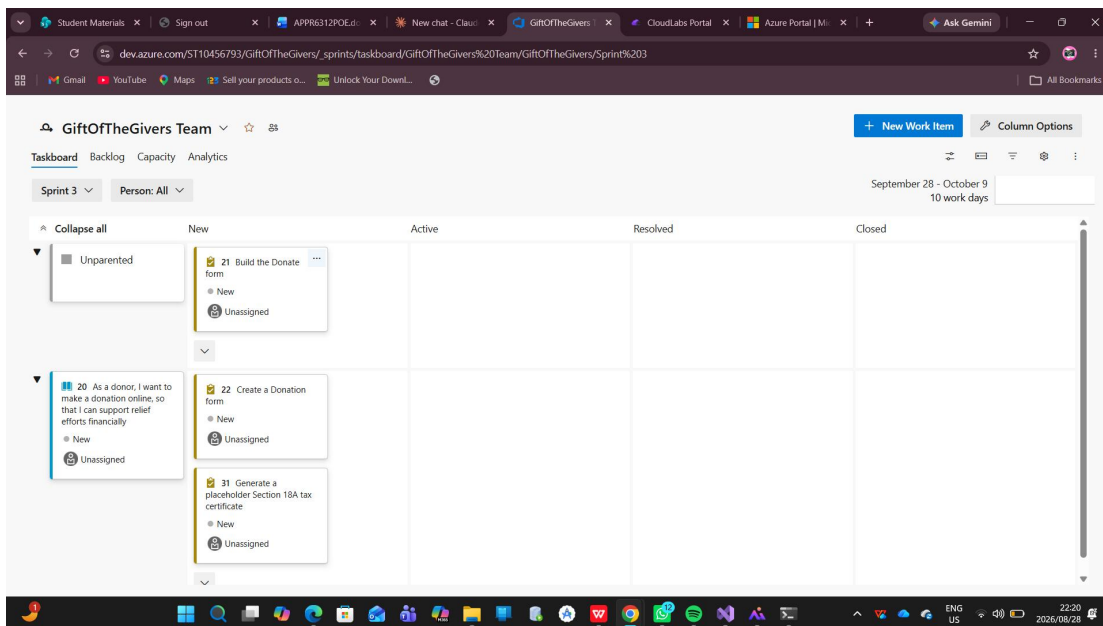
Sprint 3 — Donation Management

Goal: Allow donations to be recorded and monitored.

User Stories:

- Record donation
- View donations
- Filter donations

Deliverable: A functional donation-management module.



Sprint 4 — Relief Project Management

Goal: Allow disaster-relief projects and volunteers to be managed.

User Stories:

- Create relief project
- View relief projects
- Assign volunteers

Deliverable: A functional relief-project-management module.

Sprint 5 — Testing and Improvements

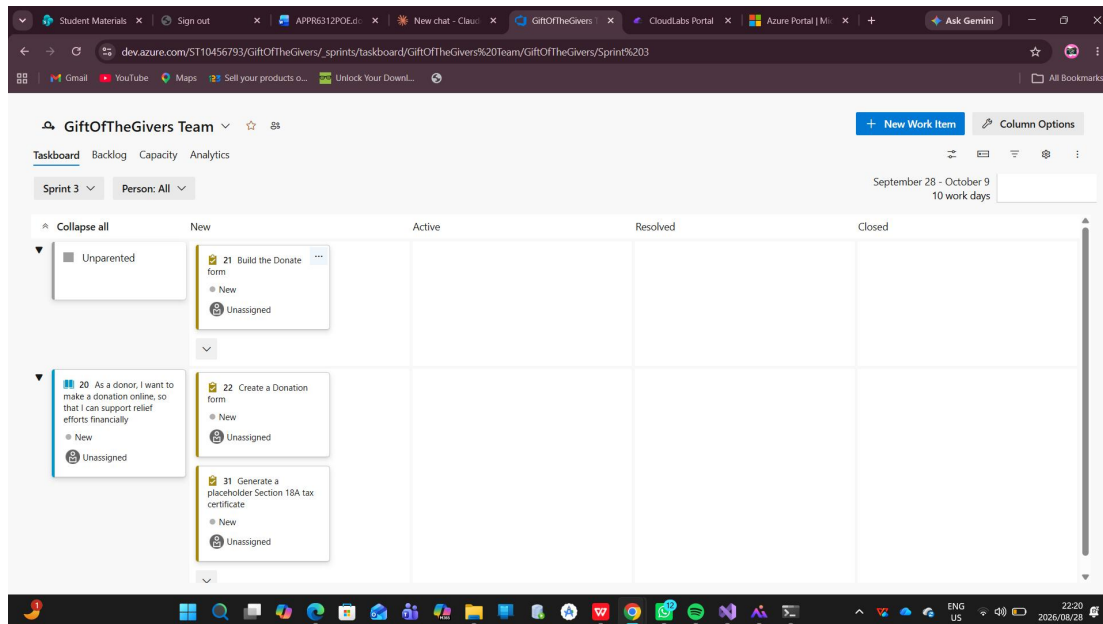
Goal: Ensure that the application is reliable and ready for demonstration.

Activities:

- Functional testing
- Database testing
- Fixing defects
- Improving usability
- Final integration

Deliverable: A tested and improved application prototype.

The application was divided into modules using the same area paths described in Section 1.1 (User Management, Volunteer Management, Donation Management, and Relief Project Management), allowing the team to organise work according to different parts of the application.



Section B: Database Design

2.1 Why a Good Database Design Is Critical

Scalability

When too many volunteers use the database at once, a poorly designed database will slow down, run out of memory, and eventually crash. Volunteers would experience lost connections, error messages, or lost data. A well-designed database, by contrast, can handle sudden spikes or large numbers of volunteers registering at once without a drop in performance.

Data Integrity

Duplicate volunteer registrations or incorrect stock counts can be prevented, even during high-stress moments, by using database constraints such as primary keys, foreign keys, and proper normalisation. This keeps the relationships between tables — volunteers, resources, locations, and incidents — consistent and trustworthy.

Efficiency

Indexing, normalisation, and referential integrity speed up crisis decisions by making data queries fast and ensuring the facts returned can be trusted. For example, a speedy decision might involve checking which volunteers with medical training are near a given location at a time of need.

Referential Integrity

Referential integrity enforces strict rules between connected data tables. Volunteers are linked to specific skills, and resources are linked to the correct warehouse, so this information stays accurate and staff can trust the data and act on it immediately.

2.2 Creating the Database Schema

Why these entities?

Volunteers, Incidents, Relief Centres, and Resources map directly to the Foundation's actual operations: the organisation must monitor who is available to assist (Volunteers), the crisis being addressed (Incidents), the locations from which aid is provided (Relief Centres), and the items being distributed (Resources).

A separate Volunteer Assignments entity was included because a volunteer may assist with several incidents over time, and each incident requires multiple volunteers — a simple one-to-many relationship would not be sufficient to represent that.

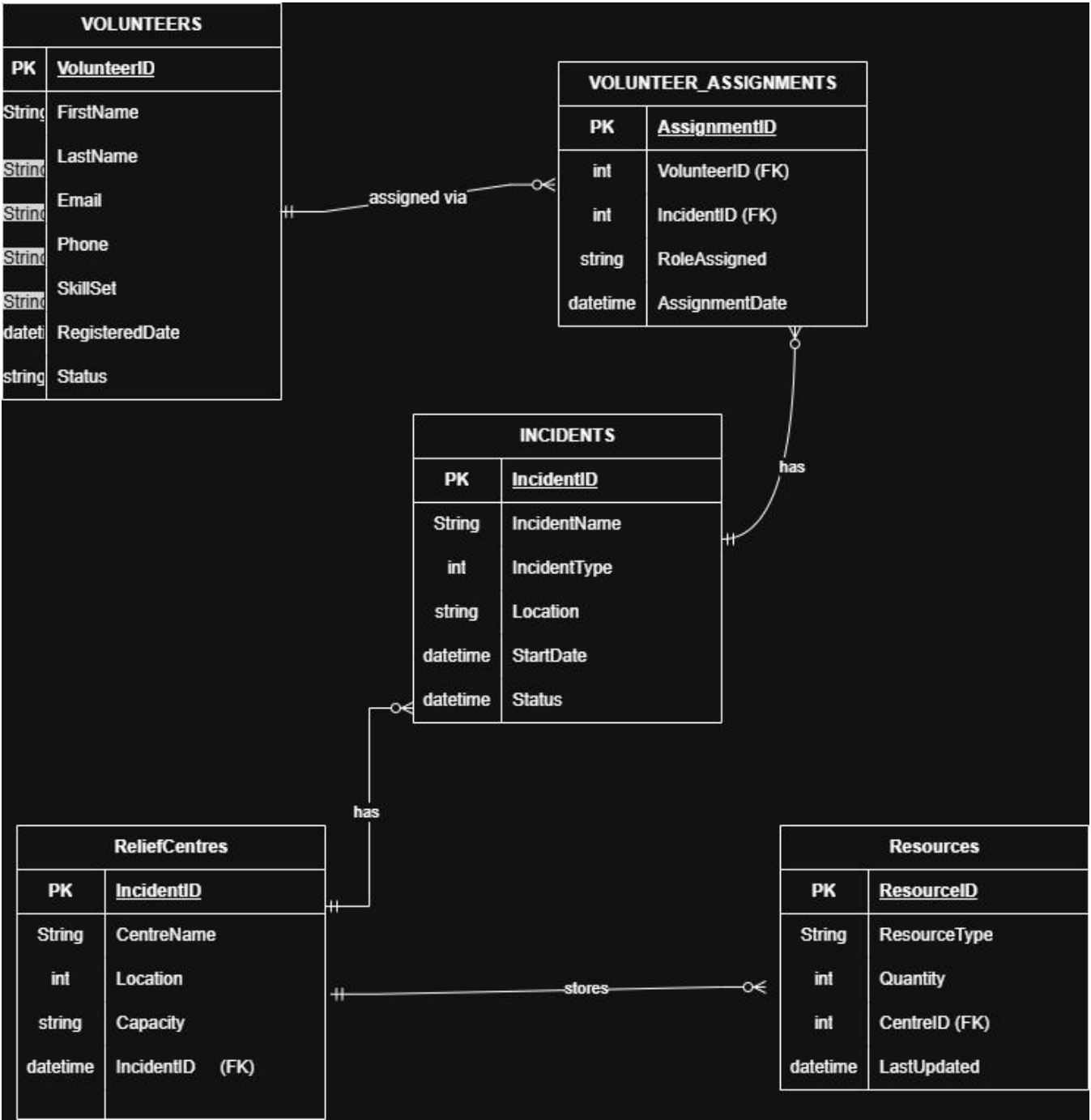
Why these relationships?

- Incidents → Relief Centres (1:M): a single disaster can lead to the establishment of multiple response locations, such as several distribution hubs for one flood.
- Relief Centres → Resources (1:M): every centre operates its own inventory separately, so resources are linked to a particular centre rather than to the overall incident.
- Volunteers ↔ Incidents (M:M, via Volunteer Assignments): this reflects real-life adaptability — volunteers shift between different crises, and each crisis requires help from many volunteers.

How this helps the Foundation in practice

This structure allows staff to quickly answer operational questions during emergencies, such as "Who are the volunteers assigned to the KZN floods?" or "What is the remaining water stock at the Durban hub?" — all while avoiding data duplication and maintaining consistent records. It also allows the system to expand as the Foundation's operations grow.

(Entity Diagram)



2.3 Optimising Performance

Indexing on high-traffic lookup columns

In addition to the primary keys, indexes will be created on `Incidents.Location`, `Incidents.Status`, and `VolunteerAssignments.IncidentID / VolunteerID`. These columns are essential for staff to filter frequently during a live crisis — for example, "Show all active incidents in KwaZulu-Natal" or "Show all volunteers assigned to Incident #4." Without these indexes, queries would require full table scans as the data grows.

Query optimisation for frequent joins

The typical query pattern involves joining `Volunteers`, `VolunteerAssignments`, and `Incidents` to determine assignments. A covering index on `VolunteerAssignments(IncidentID, VolunteerID)` makes this join faster without needing to access the base tables directly.

Section C: Technology Choice

3.1 .NET Framework vs .NET Core

.NET Framework is a development platform from Microsoft that runs mainly on Windows, and is often used for applications such as ASP.NET and Windows Forms.

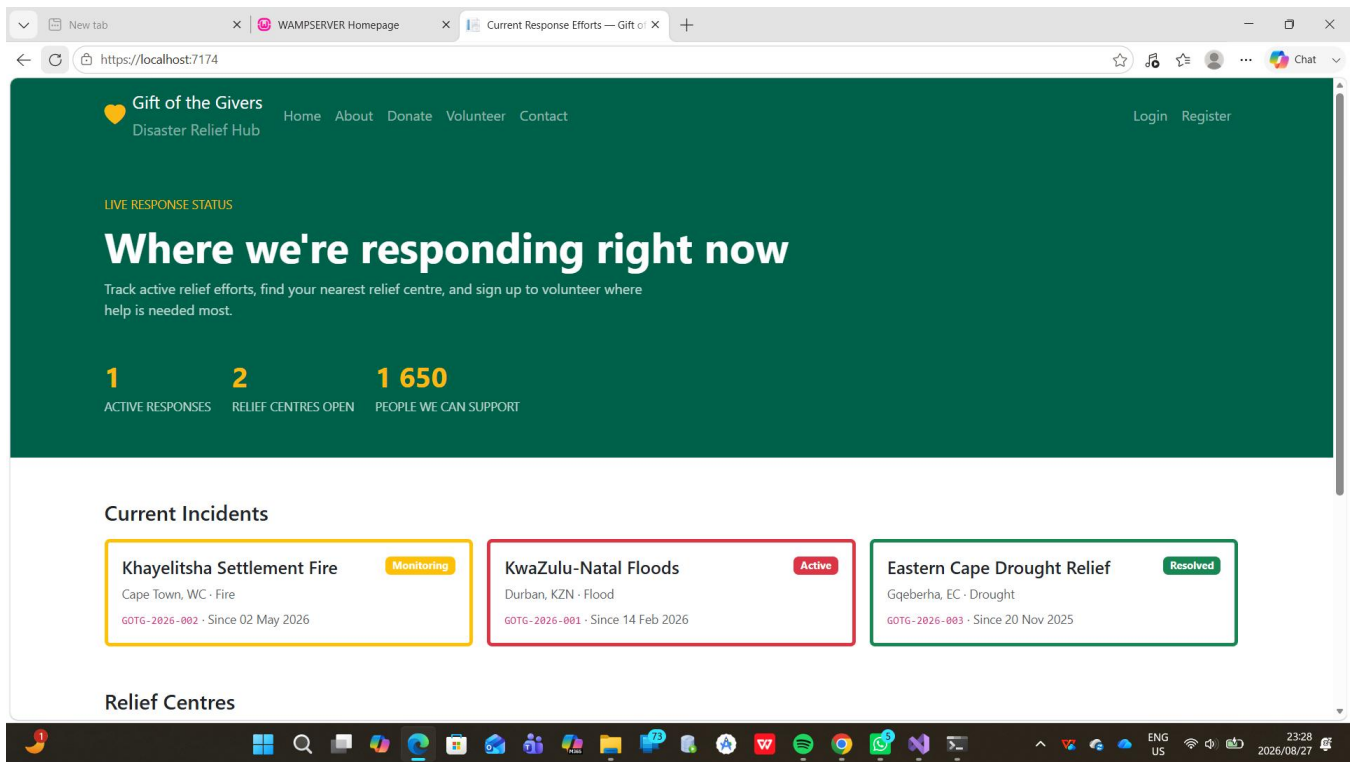
.NET Core is a cross-platform version of .NET. It can run on Windows, Linux and macOS, which makes it more flexible for development.

For an application that will run in Azure, .NET Core is the better choice. It offers portability, because the same application can run on multiple operating systems; it delivers strong performance; and it was built with cloud and web development in mind.

.NET Core also works well with Azure specifically — it supports cloud-based development, containers, scaling, and modern deployment pipelines. For these reasons, .NET Core was chosen over .NET Framework for this Azure-hosted application: it brings portability, performance, and readiness for the cloud.

3.2





3.3

